Bài 10

Máy vector hỗ trợ

10.1. Giới thiệu

Máy vector hỗ trợ (support vector machine, SVM) là một trong những thuật toán phân loại phổ biến và hiệu quả. Ý tưởng đứng sau SVM khá đơn giản, nhưng để giải bài toán tối ưu SVM, chúng ta cần kiến thức về tối ưu và đối ngẫu.

Trước khi đi vào phần ý tưởng chính của SVM, chúng ta cùng ôn lại kiến thức về hình học giải tích trong chương trình phổ thông.

10.1.1. Khoảng cách từ một điểm tới một siêu mặt phẳng

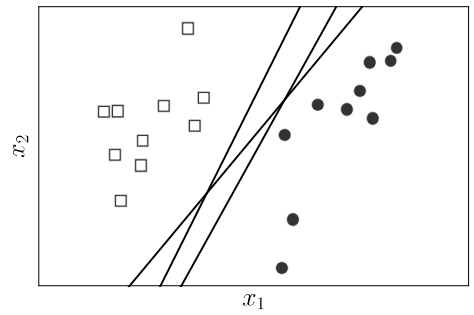
Trong không gian hai chiều, khoảng cách từ một điểm có toạ độ (x_0, y_0) tới đường thẳng có phương trình $w_1x + w_2y + b = 0$ được xác định bởi

$$\frac{|w_1x_0 + w_2y_0 + b|}{\sqrt{w_1^2 + w_2^2}}$$

Trong không gian ba chiều, khoảng cách từ một điểm có toạ độ (x_0, y_0, z_0) tới một mặt phẳng có phương trình $w_1x + w_2y + w_3z + b = 0$ được xác định bởi

$$\frac{|w_1x_0 + w_2y_0 + w_3z_0 + b|}{\sqrt{w_1^2 + w_2^2 + w_3^2}}$$

Hơn nữa, nếu bỏ dấu trị tuyệt đối ở tử số, ta có thể xác định được điểm đó nằm về phía nào của đường thẳng hay mặt phẳng đang xét. Những điểm làm cho biểu thức trong dấu trị tuyệt đối mang dấu dương nằm về cùng một phía (tạm gọi là *phía dương*), những điểm làm cho giá trị này mang dấu âm nằm về phía còn lại (gọi là *phía âm*). Những điểm làm cho tử số bằng không sẽ nằm trên đường thẳng/mặt phẳng phân chia.



Hình 10.1. Hai lớp dữ liệu vuông và tròn là tách biệt tuyến tính. Có vô số đường thẳng có thể phân loại chính xác hai lớp dữ liệu này (xem thêm Bài 7).

Các công thức này có thể được tổng quát lên cho trường hợp không gian d chiều. Khoảng cách từ một điểm (vector) có toạ độ $(x_{10}, x_{20}, \ldots, x_{d0})$ tới siêu phẳng $w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = 0$ được xác định bởi

$$\frac{|w_1x_{10} + w_2x_{20} + \cdots + w_dx_{d0} + b|}{\sqrt{w_1^2 + w_2^2 + \cdots + w_d^2}} = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|_2}$$

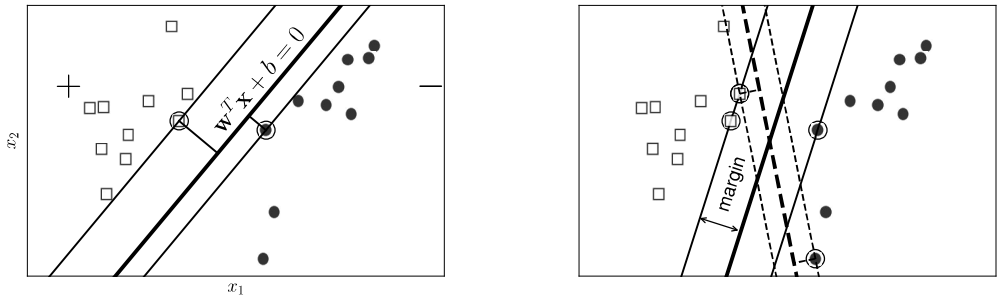
với $\mathbf{x}_0 = [x_{10}, x_{20}, \ldots, x_{d0}]^T, \mathbf{w} = [w_1, w_2, \ldots, w_d]^T$.

10.1.2. Nhắc lại bài toán phân loại hai lớp dữ liệu

Xin nhắc lại bài toán phân loại đã đề cập trong Bài 7 (PLA). Giả sử có hai lớp dữ liệu được mô tả bởi các vector đặc trưng trong không gian nhiều chiều. Hơn nữa, hai lớp dữ liệu này là tách biệt tuyến tính, tức tồn tại một siêu phẳng phân chia chính xác hai lớp đó. Hãy tìm một siêu phẳng sao cho tất cả các điểm thuộc một lớp nằm về cùng một phía của siêu phẳng đó và ngược phía với toàn bộ các điểm thuộc lớp còn lại. Chúng ta đã biết rằng, thuật toán PLA có thể thực hiện được việc này nhưng PLA có thể cho vô số nghiệm như Hình 10.1.

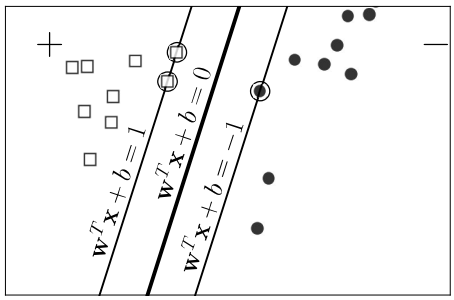
Có một câu hỏi được đặt ra: Trong vô số các mặt phân chia đó, đâu là mặt tốt nhất? Trong ba đường thẳng minh họa trong Hình 10.1, có hai đường thẳng khá lệch về phía lớp hình tròn. Điều này có thể khiến nhiều điểm hình tròn chưa nhìn thấy bị phân loại lỗi thành điểm hình vuông. Liệu có cách nào tìm được đường phân chia sao cho đường này không lệch về một lớp không?

Để trả lời câu hỏi này, chúng ta cần tìm một tiêu chuẩn để đo sự *lệch* về mỗi lớp của đường phân chia. Gọi khoảng cách nhỏ nhất từ một điểm thuộc một lớp tới đường phân chia là *lề* (margin). Ta cần tìm một đường phân chia sao cho lề của hai lớp là như nhau đối với đường phân chia đó. Hơn nữa, độ rộng của lề càng lớn thì khả năng xảy ra phân loại lỗi càng thấp. Bài toán tối ưu trong SVM chính là bài toán đi tìm đường phân chia sao cho lề rộng nhất. Đây cũng là lý do SVM còn được gọi là *bộ phân loại lề lớn nhất* (maximum margin classifier). Nguồn gốc tên gọi *máy vector hỗ trợ* sẽ sớm được làm sáng tỏ.



(a) **(b)**

Hình 10.2. Ý tưởng của SVM. Lề của một lớp được định nghĩa là khoảng cách từ các điểm gần nhất của lớp đó tới mặt phân chia. Lề của hai lớp phải bằng nhau và lớn nhất có thể.



Hình 10.3. Giả sử mặt phân chia có phương trình $\mathbf{w}^T\mathbf{x} + b = 0$. Không mất tính tổng quát, bằng cách nhân các hệ số $\mathbf{w}$ và b với các hằng số phù hợp, ta có thể giả sử rằng điểm gần nhất của lớp vuông tới mặt này thoả mãn $\mathbf{w}^T\mathbf{x}+b = 1$. Khi đó, điểm gần nhất của lớp tròn thoả mãn $\mathbf{w}^T\mathbf{x} + b = -1$.

10.2. Xây dựng bài toán tối ưu cho máy vector hỗ trợ

Giả sử dữ liệu trong tập huấn luyện là các cặp (vector đặc trưng, nhãn): $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$ nhãn bằng +1 hoặc -1 và N là số điểm dữ liệu. Không mất tính tổng quát, giả sử các điểm vuông có nhãn là 1, các điểm tròn có nhãn là -1 và siêu phẳng $\mathbf{w}^T\mathbf{x} + b = 0$ là mặt phân chia hai lớp (Hình 10.3). Ngoài ra, lớp hình vuông nằm về phía dương, lớp hình tròn nằm về phía âm của mặt phân chia. Nếu xảy ra điều ngược lại, ta chỉ cần đổi dấu của $\mathbf{w}$ và b. Bài toán tối ưu trong SVM sẽ là bài toán đi tìm các tham số mô hình $\mathbf{w}$ và b.

Với cặp dữ liệu $(\mathbf{x}_n, y_n)$ bất kỳ, khoảng cách từ $\mathbf{x}_n$ tới mặt phân chia là $\frac{y_n(\mathbf{w}^T\mathbf{x}_n+b)}{\|\mathbf{w}\|_2}$. Điều này xảy ra ta đã giả sử y_n cùng dấu với phía của $\mathbf{x}_n$. Từ đó suy ra y_n cùng dấu với $(\mathbf{w}^T\mathbf{x}_n + b)$ và tử số luôn là một đại lượng không âm. Với mặt phân chia này, lề được tính là khoảng cách gần nhất từ một điểm (trong cả hai lớp, vì cuối cùng lề của hai lớp bằng nhau) tới mặt phân chia:

$$\text{lề} = \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}$$

Bài toán tối ưu của SVM đi tìm $\mathbf{w}$ và b sao cho lề đạt giá trị lớn nhất:

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \left\{ \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \right\} = \arg\max_{\mathbf{w},b} \left\{ \frac{1}{\|\mathbf{w}\|_2} \min_n y_n(\mathbf{w}^T\mathbf{x}_n + b) \right\} \tag{10.1}$$

Nếu ta thay vector trọng số $\mathbf{w}$ bởi $k\mathbf{w}$ và b bởi kb trong đó k là một hằng số dương bất kỳ thì mặt phân chia không thay đổi, tức khoảng cách từ từng điểm đến mặt phân chia không đổi, tức lề không đổi. Vì vậy, ta có thể giả sử:

$$y_m(\mathbf{w}^T\mathbf{x}_m + b) = 1$$

với những điểm nằm gần mặt phân chia nhất (được khoanh tròn trong Hình 10.3).

Như vậy, với mọi n ta luôn có

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1$$

Bài toán tối ưu (10.1) có thể được đưa về bài toán tối ưu ràng buộc có dạng

$$\begin{gathered} (\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|_2} \\ \text{thoả mãn: } y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \forall n = 1, 2, \ldots, N \end{gathered} \tag{10.2}$$

Bằng một biến đổi đơn giản, ta có thể tiếp tục đưa bài toán này về dạng

$$\begin{gathered} (\mathbf{w}, b) = \arg\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \\ \text{thoả mãn: } 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \forall n = 1, 2, \ldots, N \end{gathered} \tag{10.3}$$

Ở đây, ta đã lấy nghịch đảo hàm mục tiêu, bình phương nó để được một hàm khả vi, và nhân với $\frac{1}{2}$ để biểu thức đạo hàm đẹp hơn.

Trong bài toán (10.3), hàm mục tiêu là một chuẩn – có dạng toàn phương. Các hàm bất phương trình ràng buộc là affine. Vậy bài toán (10.3) là một bài toán quy hoạch toàn phương. Hơn nữa, hàm mục tiêu là lồi chặt vì $\|\mathbf{w}\|_2^2 = \mathbf{w}^T\mathbf{I}\mathbf{w}$ và $\mathbf{I}$ là ma trận đơn vị – một ma trận xác định dương. Từ đây có thể suy ra nghiệm của SVM là duy nhất.

Tới đây, bài toán này có thể giải được bằng các công cụ hỗ trợ giải quy hoạch toàn phương, ví dụ CVXOPT. Tuy nhiên, việc giải bài toán này trở nên phức tạp khi số chiều d của không gian dữ liệu và số điểm dữ liệu N lớn. Thay vào đó, người ta thường giải bài toán đối ngẫu của bài toán này. Thứ nhất, bài toán đối ngẫu có những tính chất thú vị khiến nó được giải một cách hiệu quả hơn. Thứ hai, trong quá trình xây dựng bài toán đối ngẫu, người ta thấy rằng SVM có thể được áp dụng cho những bài toán mà dữ liệu không nhất thiết tách biệt tuyến tính, như chúng ta sẽ thấy sau này.

Xác định lớp cho một điểm dữ liệu mới

Sau khi đã tìm được mặt phân chia $\mathbf{w}^T\mathbf{x} + b = 0$, nhãn của một điểm bất kỳ sẽ được xác định đơn giản bằng

$$\text{class}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T\mathbf{x} + b)$$

10.3. Bài toán đối ngẫu của máy vector hỗ trợ

Bài toán tối ưu (10.3) là một bài toán lồi. Chúng ta biết rằng nếu một bài toán lồi thoả mãn tiêu chuẩn Slater thì đối ngẫu mạnh xảy ra. Ngoài ra, nếu đối ngẫu mạnh thoả mãn thì nghiệm của bài toán chính là nghiệm của hệ điều kiện KKT.

10.3.1. Kiểm tra tiêu chuẩn Slater

Trong bước này, chúng ta sẽ chứng minh bài toán tối ưu (10.3) thoả mãn điều kiện Slater. Điều kiện Slater nói rằng, nếu tồn tại $\mathbf{w}, b$ thoả mãn

$$1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) < 0, \quad \forall n = 1, 2, \ldots, N$$

thì đối ngẫu mạnh cũng thoả mãn. Việc kiểm tra điều kiện này không quá phức tạp. Vì luôn có một siêu phẳng phân chia hai lớp dữ liệu tách biệt tuyến tính nên tập khả thi của bài toán tối ưu (10.3) khác rỗng. Điều này cũng có nghĩa là luôn tồn tại cặp $(\mathbf{w}_0, b_0)$ sao cho:

$$1 - y_n(\mathbf{w}_0^T\mathbf{x}_n + b_0) \leq 0, \quad \forall n = 1, 2, \ldots, N \tag{10.4}$$

$$\Leftrightarrow 2 - y_n(2\mathbf{w}_0^T\mathbf{x}_n + 2b_0) \leq 0, \quad \forall n = 1, 2, \ldots, N \tag{10.5}$$

Vậy chỉ cần chọn $\mathbf{w}_1 = 2\mathbf{w}_0$ và $b_1 = 2b_0$, ta sẽ có:

$$1 - y_n(\mathbf{w}_1^T\mathbf{x}_n + b_1) \leq -1 < 0, \quad \forall n = 1, 2, \ldots, N$$

Điều này chỉ ra rằng $(\mathbf{w}_1, b_1)$ là một điểm khả thi chặt. Từ đó suy ra điều kiện Slater thoả mãn.

10.3.2. Hàm Lagrange của bài toán tối ưu

Hàm Lagrange của bài toán (10.3) là

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + \sum_{n=1}^{N} \lambda_n(1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)) \tag{10.6}$$

với $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \ldots, \lambda_N]^T$ và $\lambda_n \geq 0, \ \forall n = 1, 2, \ldots, N$.

10.3.3. Hàm đối ngẫu Lagrange

Theo định nghĩa, hàm đối ngẫu Lagrange là

$$g(\boldsymbol{\lambda}) = \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda})$$

với $\boldsymbol{\lambda} \succeq 0$. Việc tìm giá trị nhỏ nhất của hàm này theo $\mathbf{w}$ và b có thể được thực hiện bằng cách giải hệ phương trình đạo hàm của $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda})$ theo $\mathbf{w}$ và b bằng 0:

$$\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \mathbf{w} - \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n = \mathbf{0} \Rightarrow \mathbf{w} = \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n \tag{10.7}$$

$$\nabla_{b}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \sum_{n=1}^{N} \lambda_n y_n = 0 \tag{10.8}$$

Thay (10.7) và (10.8) vào (10.6) ta thu được $g(\boldsymbol{\lambda})$[1]:

$$g(\boldsymbol{\lambda}) = \sum_{n=1}^{N} \lambda_n - \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m \tag{10.9}$$

Hàm $g(\boldsymbol{\lambda})$ trong (10.9) là hàm số quan trọng nhất của SVM.

Ta có thể viết lại $g(\boldsymbol{\lambda})$ dưới dạng[2]

$$g(\boldsymbol{\lambda}) = -\frac{1}{2}\boldsymbol{\lambda}^T \mathbf{V}^T \mathbf{V} \boldsymbol{\lambda} + \mathbf{1}^T \boldsymbol{\lambda}. \tag{10.10}$$

với $\mathbf{V} = \left[y_1\mathbf{x}_1, y_2\mathbf{x}_2, \ldots, y_N\mathbf{x}_N\right]$ và $\mathbf{1} = [1, 1, \ldots, 1]^T$.

Nếu đặt $\mathbf{K} = \mathbf{V}^T\mathbf{V}$ thì $\mathbf{K}$ là một ma trận nửa xác định dương. Thật vậy, với mọi vector $\boldsymbol{\lambda}$ ta có $\boldsymbol{\lambda}^T\mathbf{K}\boldsymbol{\lambda} = \boldsymbol{\lambda}^T\mathbf{V}^T\mathbf{V}\boldsymbol{\lambda} = \|\mathbf{V}\boldsymbol{\lambda}\|_2^2 \geq 0$. Vậy $g(\boldsymbol{\lambda}) = -\frac{1}{2}\boldsymbol{\lambda}^T\mathbf{K}\boldsymbol{\lambda} + \mathbf{1}^T\boldsymbol{\lambda}$ là một hàm lồi.

10.3.4. Bài toán đối ngẫu Lagrange

Từ đó, kết hợp hàm đối ngẫu Lagrange và các điều kiện ràng buộc của $\boldsymbol{\lambda}$, ta sẽ thu được bài toán đối ngẫu Lagrange của bài toán (10.3):

$$\begin{aligned} \boldsymbol{\lambda} &= \arg\max_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}) \\ \text{thoả mãn: } &\boldsymbol{\lambda} \succeq 0 \\ &\sum_{n=1}^{N} \lambda_n y_n = 0 \end{aligned} \tag{10.11}$$

[1] Phần chứng minh coi như một bài tập nhỏ cho bạn đọc.
[2] Phần chứng minh coi như một bài tập nhỏ khác cho sinh viên.

Ràng buộc thứ hai được lấy từ (10.8). Đây là một bài toán lồi vì ta đang đi tìm giá trị lớn nhất của một hàm mục tiêu lõm trên một đa diện. Hơn nữa, đây là một bài toán quy hoạch toàn phương và cũng có thể được giải bằng các thư viện như CVXOPT.

Biến tối ưu trong bài toán tối ngẫu là $\boldsymbol{\lambda}$, là một vector N chiều tương ứng với số điểm dữ liệu. Trong khi đó, số tham số phải tìm trong bài toán tối ưu chính (10.3) là $d+1$, chính là tổng số chiều của $\mathbf{w}$ và b, tức số chiều của mỗi điểm dữ liệu cộng một. Trong rất nhiều trường hợp, số điểm dữ liệu trong tập huấn luyện lớn hơn số chiều dữ liệu. Nếu giải trực tiếp bằng các công cụ giải quy hoạch toàn phương, bài toán đối ngẫu có thể phức tạp hơn bài toán gốc. Tuy nhiên, điểm hấp dẫn của bài toán đối ngẫu này đến từ cấu trúc đặc biệt của hệ điều kiện KKT.

10.3.5. Điều kiện KKT

Quay trở lại bài toán, vì đây là một bài toán tối ưu lồi và đối ngẫu mạnh xảy ra, nghiệm của bài toán thoả mãn hệ điều kiện KKT sau đây với biến số $\mathbf{w}, b$ và $\boldsymbol{\lambda}$:

$$1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \ \forall n = 1, 2, \ldots, N \tag{10.12}$$

$$\lambda_n \geq 0, \ \forall n = 1, 2, \ldots, N \tag{10.13}$$

$$\lambda_n(1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)) = 0, \ \forall n = 1, 2, \ldots, N \tag{10.14}$$

$$\mathbf{w} = \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n \tag{10.15}$$

$$\sum_{n=1}^{N} \lambda_n y_n = 0 \tag{10.16}$$

Trong những điều kiện trên, điều kiện lỏng lẻo bù trừ (10.14) là thú vị nhất. Từ đó ta có thể suy ra $\lambda_n = 0$ hoặc $1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) = 0$ với n bất kỳ. Trường hợp thứ hai tương đương với

$$\mathbf{w}^T\mathbf{x}_n + b = y_n. \tag{10.17}$$

Những điểm thoả mãn (10.17) chính là những điểm nằm gần mặt phân chia nhất (những điểm được khoanh tròn trong Hình 10.3). Hai đường thẳng $\mathbf{w}^T\mathbf{x}_n + b = \pm 1$ *tựa* lên các vector thoả mãn (10.17). Những vector thoả mãn (10.17) được gọi là *vector hỗ trợ*(support vector). Tên gọi *máy vector hỗ trợ* xuất phát từ đây.

Số lượng điểm thoả mãn (10.17) thường chiếm một lượng nhỏ trong số N điểm dữ liệu huấn luyện. Chỉ cần dựa trên những vector hỗ trợ này, chúng ta hoàn toàn có thể xác định được mặt phân cách cần tìm. Nói cách khác, hầu hết các λ_n bằng không, tức λ là một vector thưa. Máy vector hỗ trợ vì vậy cũng được coi là một *mô hình thưa* (sparse model). Các mô hình thưa thường có cách giải quyết hiệu quả hơn các mô hình tương tự với nghiệm *dày đặc* (dense, hầu hết các phần tử khác không). Đây là lý do thứ hai của việc bài toán đối ngẫu SVM được quan tâm nhiều hơn là bài toán chính.

Tiếp tục phân tích, với những bài toán với số điểm dữ liệu N nhỏ, ta có thể giải hệ điều kiện KKT phía trên bằng cách xét các trường hợp $\lambda_n = 0$ hoặc $\lambda_n \neq 0$. Tổng số trường hợp phải xét là 2^N. Thông thường, $N > 50$ và 2^N là một con số rất lớn. Việc thử 2^N trường hợp là bất khả thi. Phương pháp thường được dùng để giải hệ này là *sequential minimal optimization* (SMO). Trong phạm vi cuốn sách, chúng ta sẽ không đi sâu tiếp vào việc giải hệ KKT như thế nào.

Trong phần tiếp theo chúng ta sẽ giải bài toán tối ưu (10.11) qua một ví dụ nhỏ bằng CVXOPT, và trực tiếp sử dụng thư viện `sklearn` để huấn luyện mô hình SVM. Sau khi tìm được $\boldsymbol{\lambda}$ từ bài toán (10.11), ta có thể suy ra $\mathbf{w}$ dựa vào (10.15) và b dựa vào (10.14) và (10.16). Rõ ràng ta chỉ cần quan tâm tới $\lambda_n \neq 0$.

Đặt $\mathcal{S} = \{n : \lambda_n \neq 0\}$ và $N_{\mathcal{S}}$ là số phần tử của $\mathcal{S}$. Theo (10.15), $\mathbf{w}$ được tính bằng

$$\mathbf{w} = \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m. \tag{10.18}$$

Với mỗi $n \in \mathcal{S}$, ta có

$$1 = y_n(\mathbf{w}^T \mathbf{x}_n + b) \Leftrightarrow b = y_n - \mathbf{w}^T \mathbf{x}_n.$$

Mặc dù hoàn toàn có thể suy ra b từ một cặp $(\mathbf{x}_n, y_n)$ nếu đã biết $\mathbf{w}$, một phiên bản tính b khác thường được sử dụng và có phần ổn định hơn trong tính toán là trung bình cộng[3] của các b tính được theo mỗi $n \in \mathcal{S}$

$$b = \frac{1}{N_{\mathcal{S}}} \sum_{n \in \mathcal{S}} (y_n - \mathbf{w}^T \mathbf{x}_n) = \frac{1}{N_{\mathcal{S}}} \sum_{n \in \mathcal{S}} \left(y_n - \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x}_n \right) \tag{10.19}$$

Để xác định một điểm $\mathbf{x}$ thuộc vào lớp nào, ta cần tìm dấu của biểu thức

$$\mathbf{w}^T \mathbf{x} + b = \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x} + \frac{1}{N_{\mathcal{S}}} \sum_{n \in \mathcal{S}} \left(y_n - \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x}_n \right).$$

Biểu thức này phụ thuộc vào cách tính tích vô hướng giữa $\mathbf{x}$ và từng $\mathbf{x}_m \in \mathcal{S}$.

10.4. Lập trình tìm nghiệm cho máy vector hỗ trợ

Trong mục này, ta sẽ tìm nghiệm của SVM bằng hai cách khác nhau. Cách thứ nhất dựa trên bài toán (10.11) với nghiệm tìm được theo các công thức (10.19) và (10.18). Cách làm này giúp chứng minh tính đúng đắn của các công thức đã xây dựng. Cách thứ hai sử dụng trực tiếp thư viện `sklearn`, giúp sinh viên làm quen với việc áp dụng SVM vào dữ liệu thực tế.

[3] Việc lấy trung bình này giống cách đo trong các thí nghiệm vật lý. Để đo một đại lượng, người ta thường thực hiện việc đo nhiều lần rồi lấy kết quả trung bình để tránh sai số. Ở đây, về mặt toán học, b phải như nhau theo mọi cách tính. Tuy nhiên, khi tính toán bằng máy tính, chúng ta có thể gặp các sai số nhỏ. Việc lấy trung bình sẽ làm giảm sai số đó.

10.4.1. Tìm nghiệm theo công thức

Trước tiên ta khai báo các thư viện và tạo dữ liệu giả (dữ liệu này được sử dụng trong các hình từ đầu bài. Ta thấy rằng hai lớp dữ liệu tách biệt tuyến tính):

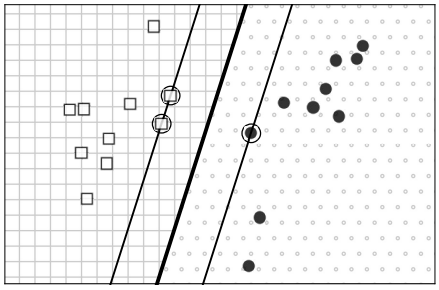
```
from __future__ import print_function
import numpy as np
np.random.seed(22)
# simulated samples
means = [[2, 2], [4, 2]]
cov = [[.3, .2], [.2, .3]]
N = 10
X0 = np.random.multivariate_normal(means[0], cov, N) # blue class data
X1 = np.random.multivariate_normal(means[1], cov, N) # red class data
X = np.concatenate((X0, X1), axis = 0)               # all data
y = np.concatenate((np.ones(N), -np.ones(N)), axis = 0) # label
# solving the dual problem (variable: lambda)
from cvxopt import matrix, solvers
V = np.concatenate((X0, -X1), axis = 0) # V in the book
Q = matrix(V.dot(V.T))
p = matrix(-np.ones((2*N, 1))) # objective function 1/2 lambda^T*Q*lambda -
    1^T*lambda
# build A, b, G, h
G = matrix(-np.eye(2*N))
h = matrix(np.zeros((2*N, 1)))
A = matrix(y.reshape(1, -1))
b = matrix(np.zeros((1, 1)))
solvers.options['show_progress'] = False
sol = solvers.qp(Q, p, G, h, A, b)
l = np.array(sol['x']) # solution lambda

# calculate w and b
w = Xbar.T.dot(l)
S = np.where(l > 1e-8)[0] # support set, 1e-8 to avoid small value of l.
b = np.mean(y[S].reshape(-1, 1) - X[S,:].dot(w))
print('Number of suport vectors = ', S.size)
print('w = ', w.T)
print('b = ', b)
```

Kết quả:

```
Number of suport vectors =  3
w =  [[-2.00984382  0.64068336]]
b =  4.66856068329
```

Như vậy trong số 20 điểm dữ liệu của cả hai lớp, chỉ có ba điểm đóng vai trò vector hỗ trợ. Ba điểm này giúp tính `w` và `b`. Đường thẳng phân chia tìm được có màu đen đậm và được minh hoạ trong Hình 10.4. Hai đường đen mảnh thể hiện đường thẳng tựa lên các vector hỗ trợ được khoanh tròn.

Hình vẽ và mã nguồn trong bài có thể được tìm thấy tại https://goo.gl/VKBgVG.



Hình 10.4. Minh hoạ nghiệm tìm được bởi SVM. Tất cả các điểm nằm trong vùng có nền kẻ ô sẽ được phân vào cùng lớp với các điểm vuông. Điều tương tự xảy ra với các điểm tròn nằm trên nền dấu chấm.

10.4.2. Tìm nghiệm theo thư viện

Chúng ta sẽ sử dụng `sklearn.svm.SVC`[4]. Sinh viên có thể tham khảo thêm thư viện libsvm được viết trên ngôn ngữ C, có API cho Python và Matlab:

```
# solution by sklearn
from sklearn.svm import SVC

model = SVC(kernel = 'linear', C = 1e5) # just a big number
model.fit(X, y)

w = model.coef_
b = model.intercept_
print('w = ', w)
print('b = ', b)
```

Kết quả:

```
w =  [[-2.00971102  0.64194082]]
b =  [ 4.66595309]
```

Kết quả này thống nhất với kết quả tìm được ở mục trước. Có rất nhiều tuỳ chọn cho `SVC`, trong đó có thuộc tính `kernel`, chúng ta sẽ tìm hiểu sau.

10.5. Tóm tắt

- Nếu hai lớp dữ liệu tách biệt tuyến tính, có vô số các siêu phẳng phân chia hai lớp đó. Khoảng cách gần nhất từ một điểm dữ liệu tới siêu phẳng này được gọi là lề.

[4] SVC là viết tắt của *bộ phân loại vector hỗ trợ* (support vector classifier).

- SVM là bài toán đi tìm mặt phân cách sao cho lề của hai lớp bằng nhau và lớn nhất, đồng nghĩa với việc các điểm dữ liệu có một khoảng cách an toàn tới mặt phân chia.

- Bài toán tối ưu trong SVM là một bài toán quy hoạch toàn phương với hàm mục tiêu lồi chặt. Vì vậy, cực tiểu địa phương cũng là cực tiểu toàn cục của bài toán.

- Mặc dù có thể trực tiếp giải SVM qua bài toán chính, người ta thường giải bài toán đối ngẫu. Bài toán đối ngẫu cũng là một bài toán quy hoạch toàn phương nhưng nghiệm là các vector thưa nên có những phương pháp giải hiệu quả hơn. Ngoài ra, bài toán đối ngẫu có những tính chất thú vị sẽ được thảo luận sau.

10.6. Câu hỏi ôn tập bài 10

Câu 1. SVM là gì và được sử dụng cho mục đích gì?

Gợi ý: SVM là một thuật toán học có giám sát trong học máy, được sử dụng để giải quyết các bài toán phân loại và hồi quy. SVM tìm ra đường phân chia tốt nhất giữa các điểm dữ liệu và sử dụng nó để dự đoán nhãn của các điểm dữ liệu mới.

Câu 2. SVM hoạt động như thế nào?

Gợi ý: SVM tìm ra một siêu phẳng (hyperplane) trong không gian đặc trưng của dữ liệu, mà tách các điểm dữ liệu của các lớp khác nhau sao cho khoảng cách từ các điểm dữ liệu đến siêu phẳng đó là lớn nhất. SVM cố gắng tối đa hóa khoảng cách giữa siêu phẳng và các điểm gần nhất của từng lớp dữ liệu, gọi là các vector hỗ trợ (support vectors).

Câu 3. SVM sử dụng các độ đo nào để đo khoảng cách giữa siêu phẳng và các điểm dữ liệu?

Gợi ý: SVM sử dụng độ đo Euclidean (khoảng cách Euclid) để đo khoảng cách giữa siêu phẳng và các điểm dữ liệu.

Câu 4. SVM sử dụng các kernel nào để xử lý dữ liệu phi tuyến?

Gợi ý: SVM sử dụng các kernel như kernel tuyến tính, kernel đa thức, kernel RBF (radial basis function) để xử lý dữ liệu phi tuyến.

Câu 5. SVM được huấn luyện như thế nào?

Gợi ý: SVM được huấn luyện bằng cách tối ưu hóa một hàm mất mát dựa trên các điểm hỗ trợ (support vectors). Hàm mất mát này bao gồm hai thành phần:

một thành phần tối thiểu hóa khoảng cách từ các điểm dữ liệu đến siêu phẳng và một thành phần kiểm soát sự phân tách không hoàn hảo của dữ liệu.

Câu 6. SVM có thể được sử dụng trong bài toán hồi quy không?

Gợi ý: Có, SVM có thể được sử dụng trong bài toán hồi quy. SVM hồi quy cũng tìm kiếm siêu phẳng tốt nhất để dự đoán giá trị đầu ra của các điểm dữ liệu mới.

Câu 7. SVM có nhược điểm gì?

Gợi ý: Phức tạp tính toán: SVM có thể trở nên rất phức tạp khi xử lý các tập dữ liệu lớn. Điều này có thể dẫn đến thời gian tính toán lâu hơn và sử dụng nhiều tài nguyên tính toán hơn.

Khó cài đặt: SVM có nhiều siêu tham số phải được cài đặt một cách thích hợp để có thể hoạt động tốt. Việc chọn các giá trị tham số phù hợp có thể là một thách thức trong một số trường hợp.

Nhạy cảm với nhiễu: Nếu tập dữ liệu chứa nhiễu hoặc dữ liệu bị sai lệch, SVM có thể bị ảnh hưởng nhiều và có thể cho kết quả không tốt.

Không phù hợp với dữ liệu có số chiều lớn: Khi số chiều của tập dữ liệu tăng lên, việc tìm ra siêu phẳng tối ưu trở nên phức tạp và tốn nhiều thời gian tính toán. Trong các trường hợp này, các phương pháp giảm chiều dữ liệu có thể được sử dụng trước khi áp dụng SVM.

10.7. BÀI TẬP CUỐI CHƯƠNG

Bài 1. Xây dựng một bộ phân loại cho tập dữ liệu MNIST có độ chính xác trên 97% trên tập kiểm tra.

Bài 2. Hãy viết một hàm để dịch chuyển một tấm ảnh MNIST theo hướng bất kỳ (trái, phải, trên, dưới) một đơn vị điểm ảnh.Sau đó, với mỗi ảnh trong tập huấn luyện, tạo bốn bản sao bị dịch chuyển (mỗi bản sao bị dịch theo một hướng) và thêm chúng vào tập huấn luyện. Cuối cùng, huấn luyện mô hình tốt nhất trên tập dữ liệu đã mở rộng này và đo độ chính xác của nó trên tập kiểm tra. (Làm quen với kỹ thuật tăng số lượng mẫu trong tập huấn luyện (data augmentation) hay mở rộng tập huấn luyện (training set expansion)).

Bài 3. Hãy làm việc với tập dữ liệu Titanic trên Kaggle.

Bài 4. Hãy xây dựng một bộ phân loại thư rác.

Bài 5. Nên sử dụng thuật toán nào để huấn luyện mô hình Hồi quy Tuyến tính, nếu tập huấn luyện có hàng triệu đặc trưng?

Bài 6. Giả sử các đặc trưng trong tập huấn luyện có các khoảng giá trị rất khác nhau. Giả định này có thể ảnh hưởng tới những thuật toán nào với mức độ ra sao? Ta có thể giải quyết vấn đề này bằng cách nào?

Bài 7. Thuật toán Hạ Gradient có thể bị kẹt trong điểm cực tiểu địa phương khi huấn luyện mô hình Hồi quy Logistic hay không?

Bài 8. Liệu tất cả các thuật toán Hạ Gradient sẽ dẫn đến cùng một mô hình, miễn là ta chạy thuật toán đủ lâu hay không?

Bài 9. Giả sử ta sử dụng thuật toán Hạ Gradient theo Batch và theo dõi lỗi kiểm định trong từng epoch, và phát hiện ra lỗi kiểm định tăng lên liên tục. Vấn đề này có thể là gì? Ta sẽ giải quyết vấn đề này bằng cách nào?

Bài 10. Liệu ta có nên dừng thuật toán Hạ Gradient theo Mini-batch ngay khi lỗi kiểm định tăng lên?

Bài 11. Thuật toán Hạ Gradient nào (trong số những thuật toán ta đã thảo luận) sẽ tới gần nghiệm tối ưu nhanh nhất? Thuật toán nào thực sự hội tụ? Ta có thể giúp các thuật toán khác cũng hội tụ bằng cách nào?

Bài 12. Giả sử ta đang sử dụng Hồi quy Đa thức. Khi theo dõi đồ thị quá trình học, ta nhận ra có một khoảng cách lớn giữa lỗi huấn luyện và lỗi kiểm định. Vấn đề gì đang xảy ra? Hãy nêu ra ba giải pháp cho vấn đề này.

Bài 13. Giả sử ta đang sử dụng Hồi quy Ridge. Ta nhận ra rằng lỗi huấn luyện và lỗi kiểm định xấp xỉ bằng nhau và đều khá cao. Liệu mô hình có đang gặp vấn đề với độ chệch lớn hay phương sai lớn không? Ta nên tăng hay giảm giá trị siêu tham số điều chuẩn α?

Bài 14. Tại sao ta nên sử dụng:
a. Hồi quy Ridge thay vì Hồi quy Tuyến tính (không sử dụng điều chuẩn)?
b. Hồi quy Lasso thay vì Ridge?
c. Elastic Net thay vì Lasso?

Bài 15. Giả sử ta muốn phân loại các bức ảnh theo nhãn ngoài trời/trong nhà và ban ngày/ban đêm. Liệu ta cần lập trình hai bộ phân loại Hồi quy Logistic hay chỉ cần một bộ phân loại Hồi quy Softmax?

Bài 16. Hãy lập trình thuật toán Hạ Gradient theo Batch với kỹ thuật dừng sớm cho Hồi quy Softmax mà không sử dụng thư viện Scikit-Learn.

Bài 17. Ý tưởng căn bản của Máy Vector Hỗ trợ là gì?

Bài 18. Vector hỗ trợ là gì?

Bài 19. Tại sao việc co giãn giá trị đầu vào lại quan trọng khi sử dụng SVM?

Bài 20. Bộ phân loại SVM có thể trả về điểm số tin cậy và xác suất khi phân loại một mẫu không?

Bài 21. Nên sử dụng biểu diễn dạng gốc hay đối ngẫu cho bài toán SVM để huấn luyện mô hình trên tập dữ liệu huấn luyện có hàng triệu mẫu và hàng trăm đặc trưng?

Bài 22. Giả sử bạn vừa huấn luyện một bộ phân loại SVM với hạt nhân RBF, nhưng mô hình có vẻ dưới khớp tập huấn luyện. Ta nên tăng hay giảm các tham số γ (gamma) và C?

Bài 23. Nên thiết lập các tham số quy hoạch toàn phương (H, f, A, và b) như thế nào để giải bài toán cho bộ phân loại SVM biên mềm với bộ giải quy hoạch toàn phương có sẵn?

Bài 24. Hãy huấn luyện mô hình LinearSVC trên một tập dữ liệu tách biệt tuyến tính. Sau đó huấn luyện mô hình SVC và SGDClassifier trên cùng tập dữ liệu đó. Hãy thử làm cho các mô hình thu được gần giống nhau.

Bài 25. Hãy huấn luyện bộ phân loại SVM trên tập dữ liệu MNIST. Do SVM là bộ phân loại nhị phân, sẽ cần sử dụng phương pháp một-còn lại để phân loại 10 chữ số. Có thể sẽ cần tinh chỉnh siêu tham số với các tập kiểm định nhỏ để tăng tốc quá trình này. Độ chính xác cao nhất có thể đạt được là bao nhiêu?

Bài 26. Hãy huấn luyện bộ hồi quy SVM trên tập dữ liệu nhà ở California.